

M43 — Make device routes work in multi-patient mode + rename button

Phase: Phase 7 follow-on (post-M42, operator-reported bug fix) **Status:** **DONE** **Blocked by:** M2 (Encounter dataclass = ControlSession via rename), M42 (inline device manager iframe — broke because of this bug)

Blocks: future shared-cart work (M44) **Estimated effort:** 0.25 day

1. Purpose

Operator-reported bug after M42:

"the manage devices should just be managed devices. I looks like it link out to V6 or to the single character in V7, a error stated that it wasn't linked to session"

Two issues:

1. **Functional bug:** The iframe-embedded ops view's device manager threw `"No active session"` on every operator action. Root cause: every operator-facing device route in `portal/devices/routes.py` called `control_session.get_active()` which (by the M2 contract) returns `None` in v7 multi-encounter rooms. The pattern was: `python sess = control_session.get_active() if sess is None or sess.id != station["session_id"]: raise HTTPException(409, "No active session.")` The `or sess.id != station["session_id"]` guard didn't help — when `sess` is `None`, the first half short-circuits with `None.id` never evaluated, but the route 409s anyway.

2. **UX cue:** The button label "Manage devices" felt like a link- out verb, which combined with the v6-styled iframe content felt like leaving v7 for the legacy ops view. The operator wanted a panel descriptor ("Managed devices" — "here are the devices being managed for this bed").

2. Structure

Files touched:

- `portal/devices/routes.py`:
- Added two helpers near the top of the file:
- `_session_for_station(station)` — resolves a device station's

session. Tries v6 singleton via `control_session.get_active()`, then falls back to the active multi-patient room's encounters dict keyed by `station["session_id"]`. Returns `None` if neither lookup finds a match.

- `_session_for_join(join)` — resolves a session by an operator-supplied join code via `control_session.get_by_join_code`, with v6 singleton fallback.
- `POST /api/device/register` — now accepts `?join=<code>` and uses `_session_for_join`. Friendly 409 message when no session is resolved (points the operator at the Per-Patient Console).
- `POST /api/device/{station_id}/inject / /clear / /advance_time / /assign` — each swapped its `get_active()` lookup for `_session_for_station(station)`. The per-station routes don't need `?join=` because the station's stored `session_id` already tells them which encounter to target.
- `GET /api/device/roster` — now accepts `?join=<code>` and scopes to that encounter's stations. Without `join` falls back to v6 singleton (returns empty roster in multi-patient mode instead of 4xx).

- `portal/static/control_ops_devices.js`:
- New helper `_joinQuery()` builds `?join=<code>` from

`window.MEDSIM2_OPS.join_code` (set by the route's bootstrap when M42's `?join=` query param was used to open the ops view). - `refreshRoster` calls `/api/device/roster' + _joinQuery()`. - The `add-device` submit calls `/api/device/register' + _joinQuery()`. - Per-station routes (assign, inject, clear) don't need `_joinQuery()` — they resolve server-side via the station's `session_id`.

- `portal/templates/encounter_console.html`:
- Button label changed: ■ Manage devices → ■ Managed devices.
- Dialog title changed: ■ Manage devices · ... → `■ Managed devices · ...`.

No new dataclass field. No schema migration. No backend API surface change. The two `?join=` query params are additive; the v6 singleton path still works identically when `?join=` is absent.

3. Uses

3.1 The bug, end-to-end

1. Operator starts a 2-encounter room. State: active multi-patient room, `control_session.get_active()` returns `None`. 2. Opens `/portal/room/encounter/{id}` (Per-Patient Console). 3. Clicks the M42 ■ **Manage devices** button → modal opens → iframe loads `/portal/control/ops?join=<jc>&embed=1`. M42's route fix ensures the ops page itself loads. 4. Operator clicks + **Add device** inside the iframe. Picks kind + model. Hits Mint QR. The JS POSTs to `/api/device/register`. 5. **Pre-M43**: route calls `get_active()`, gets `None`, raises 409 `"No active session"`. The chat-style error renders in the modal. Operator is stuck. 6. **Post-M43**: the JS appends `?join=<jc>` (from `MEDSIM2_OPS.join_code`). Route calls `_session_for_join(jc)`, finds the encounter, registers the device, returns 200 + QR.

3.2 Per-station operations

For inject/clear/advance_time/assign, the JS uses `/api/device/{station_id}/...` — no `?join=` needed. Server-side, `_session_for_station(station)` reads `station["session_id"]` (persisted in SQLite when the station was registered) and finds the encounter in the active room by that id.

3.3 v6 single-patient path

`_session_for_station` and `_session_for_join` both call `control_session.get_active()` first. In v6 single-patient mode that returns the session immediately; the multi-patient fallback branches are never taken. The v6 path is byte-for-byte unchanged in behavior.

4. Functions (exported API surface)

Symbol	Where	Purpose
<code>_session_for_station(station)</code> (new)	<code>portal/devices/routes.py</code>	Resolve a station's session via singleton OR multi-patient room lookup.
<code>_session_for_join(join)</code> (new)	same	Resolve a session by operator-supplied join code.
POST <code>/api/device/register?join=...</code> (extended)	same	Accepts <code>?join=</code> . Friendly 409 when no session resolves.

Symbol	Where	Purpose
GET /api/device/roster?join=... (extended)	same	Accepts ?join= to scope to one encounter.
POST /api/device/{sid}/{inject,clear,advance_time,assign} (internal)	same	Uses _session_for_station — no API change for callers.
_joinQuery() (new)	portal/static/control_ops_devices.js	Builds ?join=<code> from MEDSIM2_OPS.join_code.

5. Limitations

- **The per-station routes still verify `sess.id == station["session_id"]`

implicitly** — through `_session_for_station`'s lookup: if the station's stored `session_id` matches the active room's `encounters[sid]`, the helper returns that encounter; otherwise `None`. So a station that was registered against a different room (e.g. survived a server restart with a different room active) will still 409 — which is correct.

- **Shared med carts still single-patient.** M43 fixes the

session-resolution bug but doesn't address the bigger architectural ask from the original user message ("In the case of med carts, more than one encounter can be assigned to a single med cart"). That requires `DeviceStation.character_id → character_ids: list[str]` plus a cabinet bootstrap rewrite + grouped MAR view. Tracked for M44.

- **/api/device/roster without ?join in multi-patient mode

returns an empty list, not all encounters' devices.** Deliberate — the v6 ops view's roster is single-session-scoped; the v7 union-across-room view belongs on the Multi-Patient Control dashboard (already there as `/api/room/state` device counts).

- **The 409 messages now point the operator at the Per-Patient

Console.** If the operator clicks `/portal/control/ops` manually (no encounter scope), they see a 409 with a helpful hint instead of being stuck. Acceptable trade-off.

6. Test status

Test file	Asserts	Status	Last run
tests/v7/test_device_routes_multi_patient.py::test_device_register_works_via_join_in_multi_patient	The exact bug the operator hit: register with ?join=<code> returns 200 in multi-encounter mode	PASS	2026-05-27
...::test_device_register_without_join_in_multi_patient_still_errors	No ?join= AND no singleton → friendly 409 with hint	PASS	2026-05-27

Test file	Asserts	Status	Last run
...::test_device_inject_works_in_multi_patient_via_station_session_id	Per-station inject resolves via station.session_id	PASS	2026-05-27
...::test_device_clear_works_in_multi_patient	Per-station clear works	PASS	2026-05-27
...::test_device_assign_works_in_multi_patient	Per-station assign works	PASS	2026-05-27
...::test_device_advance_time_works_in_multi_patient	Per-station advance_time works	PASS	2026-05-27
...::test_device_roster_filters_by_join_in_multi_patient	Roster scoped by ?join returns ONLY that bed's stations	PASS	2026-05-27
...::test_device_roster_empty_when_no_session	No session → empty roster (200), not 409	PASS	2026-05-27
...::test_devices_js_appends_join_query_to_register_and_roster	JS source carries _joinQuery + appends it to register + roster calls	PASS	2026-05-27
...::test_encounter_console_button_label_renamed_to_managed_devices	Button reads "Managed devices"; old "Manage devices" label gone	PASS	2026-05-27
...::test_devices_dialog_title_uses_managed_devices	Modal title reads "Managed devices · ..."	PASS	2026-05-27
Full v7 suite	295 passed, 1 skipped (M20 Playwright skip — unchanged)	PASS	2026-05-27

7. Change list

Date	Author	Change	Files
2026-05-27	claude-code	Initial M43: <code>_session_for_station + _session_for_join</code> helpers; 5 device routes swapped; JS appends <code>?join=</code> from <code>MEDSIM2_OPS.join_code</code> ; button + dialog renamed to "Managed devices"; 11 new tests; 1 phrase-only update to the M42 button test	<code>portal/devices/routes.py</code> , <code>portal/static/control_ops_devices.js</code> , <code>portal/templates/encounter_console.html</code> , <code>tests/v7/test_device_routes_multi_patient.py</code> (new), <code>tests/v7/test_inline_device_manager.py</code> (assertion update)

8. Open questions / known issues

- **M44 (deferred)** — shared med carts across encounters + grouped

MAR.** The original operator message asked for `"more than one encounter can be assigned to a single med cart"` and `"the medicine administration system will list each of the assigned character patients and their assigned medication under their character name."` M43 fixed the session-bug blocker; the cart- sharing feature is a separate piece (DeviceStation model change + cabinet bootstrap + per-patient MAR sections).

- **Native inline UI?** If after M43 the operator still feels the

iframe content looks too v6-flavored despite working functionally, M44 could replace the iframe with a native inline card. The session-resolution helpers built here generalize to that path with no further change.

- **WebSocket events.** The instructor WS firehose at

`/ws/instructor` broadcasts every device event. It doesn't yet scope by join code — every encounter's ops-view iframe sees every other encounter's events. Acceptable today (just chatter); a future refinement could filter server-side.

Render this guide to PDF with `python docs/module_guides/render_pdfs.py`.